

JAVA DOCS

Core Java Basics

Syntax, Data Types, Control Flow, Arrays & Methods

Java Agentic AI — Knowledge Base Reference

1. Introduction to Java

Java is a class-based, object-oriented, platform-independent programming language created by James Gosling at Sun Microsystems (released in 1995, now owned by Oracle). Its core design philosophy is **WORA — Write Once, Run Anywhere**. Source code is compiled into an intermediate format called **bytecode** (.class files) which runs on any machine that has a Java Virtual Machine (JVM), regardless of the underlying OS or hardware.

- **Compiled + Interpreted:** javac compiles .java to .class bytecode; the JVM interprets/JIT-compiles bytecode to native machine code.
- **Strongly & statically typed:** variable types are checked at compile time.
- **Automatic memory management:** garbage collector reclaims unused heap memory.
- **Multi-threaded:** built-in support for concurrent programming.
- **Secure:** no explicit pointers, bytecode verification, sandboxed execution (classloader + security manager historically).

1.1 A Minimal Java Program

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

The class name **must** match the filename (HelloWorld.java). The main method is the JVM's entry point: it must be **public static void main(String[] args)** — public so the JVM (outside the class) can call it, static so it can be invoked without creating an instance, void because it returns nothing to the OS directly, and it accepts an array of command-line arguments.

1.2 Compilation & Execution Pipeline

Step	Command	Output
Write source	HelloWorld.java	Human-readable source code
Compile	<code>javac HelloWorld.java</code>	HelloWorld.class (bytecode)
Run	<code>java HelloWorld</code>	JVM loads class, executes main()

2. JDK vs JRE vs JVM (Quick Overview)

A dedicated document covers this in depth, but as a foundation: the **JVM** executes bytecode; the **JRE** bundles the JVM plus core libraries needed to run applications; the **JDK** bundles the JRE plus development tools (javac, javadoc, jar, debugger) needed to build applications.

3. Variables and Data Types

Java has two categories of data types: **primitive types** (stored directly, fixed size, not objects) and **reference types** (objects, arrays, interfaces — variables hold a reference/pointer to heap memory).

3.1 The Eight Primitive Types

Type	Size	Default	Range / Notes
byte	8-bit	0	-128 to 127
short	16-bit	0	-32,768 to 32,767
int	32-bit	0	-2 ³¹ to 2 ³¹ -1 (~2.1 billion)
long	64-bit	0L	-2 ⁶³ to 2 ⁶³ -1; suffix with L
float	32-bit	0.0f	IEEE 754 single precision; suffix f
double	64-bit	0.0d	IEEE 754 double precision (default for decimals)
char	16-bit	'\u0000'	Single UTF-16 character, unsigned
boolean	1 bit (JVM-dependent)	false	true or false only

3.2 Variable Declaration & Type Inference

```
int age = 30;
double price = 19.99;
final String NAME = "Claude";    // final = constant, cannot be reassigned
var counter = 0;                  // 'var' - local type inference (Java 10+)
var list = new ArrayList<String>(); // compiler infers ArrayList<String>
```

Note: 'var' only works for local variables with an initializer -- it cannot be used for fields, method parameters, or return types, and it is still statically typed (inferred at compile time, not dynamic typing).

3.3 Type Casting

Widening (implicit) conversion happens automatically when there's no data loss risk (byte -> short -> int -> long -> float -> double). **Narrowing (explicit)** conversion requires an explicit cast and can lose data or precision.

```
int i = 100;
long l = i;           // widening - automatic
double d = l;         // widening - automatic

double x = 9.78;
int y = (int) x;       // narrowing - explicit cast, y = 9 (truncated, not rounded)

int big = 130;
byte b = (byte) big;   // narrowing overflow - wraps around: b = -126
```

4. Operators

Category	Operators	Example
Arithmetic	+ - * / %	a % b (remainder)
Assignment	= += -= *= /= %=	x += 5
Relational	== != > < >= <=	a >= b
Logical	&& !	(a > 0) && (b > 0)
Bitwise	& ^ ~ << >> >>>	x >>> 2 (unsigned right shift)
Ternary	condition ? a : b	int max = (a > b) ? a : b;
instanceof	obj instanceof Type	if (obj instanceof String s) {...}

Note: && and || are 'short-circuit' operators -- the right operand is not evaluated if the result is already determined by the left operand. & and | (single) always evaluate both sides.

5. Control Flow Statements

5.1 Conditional Statements

```

if (score >= 90) {
    grade = "A";
} else if (score >= 75) {
    grade = "B";
} else {
    grade = "C";
}

// Traditional switch
switch (day) {
    case MONDAY:
    case FRIDAY:
        System.out.println("Busy day");
        break;
    default:
        System.out.println("Normal day");
}

// Modern switch expression (Java 14+)
String result = switch (day) {
    case MONDAY, FRIDAY -> "Busy day";
    case SATURDAY, SUNDAY -> "Weekend";
    default -> "Normal day";
};

```

5.2 Loops

```

for (int i = 0; i < 5; i++) { ... }           // classic for

for (String item : itemList) { ... }         // enhanced for-each

int i = 0;

```

```

while (i < 5) { i++; } // while

int j = 0;
do { j++; } while (j < 5); // do-while (executes at least once)

```

- **break** exits the nearest enclosing loop or switch immediately.
- **continue** skips to the next iteration of the loop.
- Labeled break/continue can target an outer loop: `outer: for(...) { for(...) { break outer; } }`

6. Arrays

Arrays in Java are fixed-size, homogeneous, index-based (zero-indexed) containers. They are objects on the heap even for primitive element types, and every array carries a fixed `.length` field.

```

int[] nums = new int[5]; // default-initialized to 0
int[] nums2 = {1, 2, 3, 4, 5}; // array literal
int[][] matrix = new int[3][3]; // 2D array (array of arrays)

for (int i = 0; i < nums.length; i++) {
    nums[i] = i * i;
}

// Utility methods (java.util.Arrays)
Arrays.sort(nums);
int idx = Arrays.binarySearch(nums, 9);
int[] copy = Arrays.copyOf(nums, 10);
System.out.println(Arrays.toString(nums));

```

Note: Arrays are covariant but not truly generic-safe: an `Object[]` can reference a `String[]`, which can cause an `ArrayStoreException` at runtime if you try to insert an incompatible type.

7. Methods

```

public static int add(int a, int b) {
    return a + b;
}

// Varargs - variable number of arguments
public static int sum(int... nums) {
    int total = 0;
    for (int n : nums) total += n;
    return total;
}

// Method overloading - same name, different parameter lists
public void print(String s) { ... }
public void print(int i) { ... }

```

- Java is strictly **pass-by-value**. For objects, the value passed is the reference (memory address) itself, so mutating the object's fields inside a method affects the caller's object, but reassigning the parameter to a new object does not affect the caller's reference.

- **Overloading** is resolved at compile time (static/early binding) based on argument types.
- **Recursion** is fully supported; deep recursion can throw `StackOverflowError` since each call frame consumes stack memory.

8. Strings

Strings in Java are **immutable** objects of the final class `java.lang.String`. Once created, a `String`'s character sequence can never change -- every 'modifying' method (concat, substring, replace, `toUpperCase`, etc.) returns a brand-new `String` object.

8.1 The String Pool

String literals are interned in a special memory region called the **String Constant Pool** (inside the heap since Java 7). Identical literals share the same reference.

```
String a = "hello";
String b = "hello";
System.out.println(a == b);           // true - both point to the same pooled literal

String c = new String("hello");
System.out.println(a == c);           // false - new object on the heap, outside the pool
System.out.println(a.equals(c));       // true - equals() compares content

String d = c.intern();                 // forces c's content into (or reuse from) the pool
System.out.println(a == d);           // true
```

8.2 StringBuilder & StringBuffer

Because `String` is immutable, repeated concatenation in a loop creates many intermediate objects and is $O(n^2)$. `StringBuilder` (not thread-safe, faster) and `StringBuffer` (thread-safe, synchronized methods) provide a mutable character sequence for efficient building.

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) {
    sb.append(i).append(",");
}
String result = sb.toString();

sb.reverse();
sb.insert(0, "prefix-");
sb.deleteCharAt(0);
```

8.3 Common String Methods

Method	Purpose
<code>length()</code>	Number of characters
<code>charAt(i)</code>	Character at index <code>i</code>
<code>substring(a, b)</code>	Substring from <code>a</code> (inclusive) to <code>b</code> (exclusive)
<code>equals()</code> / <code>equalsIgnoreCase()</code>	Content comparison

Method	Purpose
compareTo()	Lexicographic comparison, returns int
split(regex)	Splits into a String[] using a regex delimiter
trim() / strip()	Removes leading/trailing whitespace (strip() is Unicode-aware, Java 11+)
format()	Printf-style formatting: String.format("%s is %d", name, age)
isBlank() / isEmpty()	Checks for whitespace-only / zero-length content

9. Reading Input

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
System.out.print("Enter your name: ");
String name = sc.nextLine();
System.out.print("Enter your age: ");
int age = sc.nextInt();
sc.close();
```

Note: Mixing `nextInt()/nextDouble()` with `nextLine()` is a classic bug source: numeric read methods leave the trailing newline in the buffer, which the next `nextLine()` call immediately consumes as an empty string. Call `sc.nextLine()` once to consume the leftover newline, or use `nextLine()` everywhere and parse manually.

10. Wrapper Classes & Autoboxing

Every primitive has a corresponding wrapper class (int -> Integer, double -> Double, etc.) that provides object behavior -- useful for collections (which only hold objects), null representation, and utility methods.

```
Integer boxed = 100;           // autoboxing: int -> Integer
int unboxed = boxed;           // auto-unboxing: Integer -> int

Integer x = 127, y = 127;
System.out.println(x == y);    // true - Integer caches values -128..127

Integer p = 200, q = 200;
System.out.println(p == q);    // false - outside cache range, different objects
System.out.println(p.equals(q)); // true - always compare wrappers with equals()
```

Note: Unboxing a null wrapper (e.g. an Integer field that was never assigned) throws a `NullPointerException` the moment it's used in a primitive context -- a very common production bug.

11. Command-Line Arguments

```
public class Args {
    public static void main(String[] args) {
        for (String arg : args) {
            System.out.println(arg);
        }
    }
}
```

```
}  
// java Args foo bar 123 -> prints: foo bar 123 (all as Strings)
```

12. Comments & Javadoc

```
// single-line comment  
/* multi-line  
comment */  
/**  
 * Javadoc comment - used to generate HTML API documentation.  
 * @param x the input value  
 * @return the squared value  
 */  
public int square(int x) { return x * x; }
```

13. The static and final Keywords

13.1 static

static members belong to the class itself rather than to any individual instance. They are allocated once, in the method area/metaspace, when the class is loaded, and are shared across all objects of that class.

```
public class Counter {  
    static int totalCount = 0;    // shared across all instances  
    int id;  
  
    Counter() {  
        totalCount++;  
        id = totalCount;  
    }  
  
    static void printTotal() {    // static method - no 'this'  
        System.out.println("Total: " + totalCount);  
    }  
  
    static {                      // static initializer block  
        System.out.println("Counter class loaded");  
    }  
}
```

- Static methods cannot be overridden (they can be hidden/redeclared, resolved at compile time based on reference type).
- A static nested class does not hold an implicit reference to an outer instance (unlike a regular inner class).
- Static initializer blocks run once, in source order, when the JVM first loads the class.

13.2 final

final has three distinct meanings depending on context:

Applied to	Meaning
Variable	Value/reference cannot be reassigned after initialization (the object it points to can still be mutated)
Method	Cannot be overridden by a subclass
Class	Cannot be subclassed at all (e.g. String, Integer are final)

```
final int MAX = 100;
// MAX = 200;           // compile error

final List<String> list = new ArrayList<>();
list.add("ok");          // allowed - list contents are mutable
// list = new ArrayList<>(); // compile error - reference itself is final
```

14. Packages & Access Modifiers

Packages are namespaces that organize related classes and prevent naming collisions, mirrored by directory structure on disk (e.g. com.company.project.service).

```
package com.example.util;

import java.util.List;
import java.util.ArrayList;
// import java.util.*;    // wildcard import

public class StringUtils { ... }
```

Modifier	Same class	Same package	Subclass (diff. pkg)	World
public	Y	Y	Y	Y
protected	Y	Y	Y	N
default (none)	Y	Y	N	N
private	Y	N	N	N

15. Enums

An enum is a special class type representing a fixed set of constants. Under the hood, each enum constant is a public static final instance of the enum type, and every enum implicitly extends java.lang.Enum (so it cannot extend any other class).

```
public enum Day {
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY;
}

public enum Planet {
    MERCURY(3.3e23, 2.4e6), EARTH(5.97e24, 6.4e6);

    private final double mass;    // enums can have fields, constructors, methods
    private final double radius;
```

```

Planet(double mass, double radius) {
    this.mass = mass;
    this.radius = radius;
}

double surfaceGravity() {
    return 6.67300E-11 * mass / (radius * radius);
}
}

Day d = Day.MONDAY;
switch (d) {
    case MONDAY -> System.out.println("Start of week");
    default -> System.out.println("Other day");
}
System.out.println(d.ordinal()); // 0 (position in declaration)
System.out.println(d.name());    // "MONDAY"
Day.valueOf("TUESDAY");          // parses a String back to the enum constant

```

16. Quick Interview Recap

- Why is main() static? So the JVM can invoke it without instantiating the class first.
- Why is String immutable? Security (used in classloading, network connections), thread-safety, safe hashcode caching for use as HashMap keys, and enabling the string pool for memory efficiency.
- Difference between == and equals()? == compares references (or primitive values); equals() compares logical content (and can be overridden).
- Difference between StringBuilder and StringBuffer? StringBuffer is synchronized (thread-safe but slower); StringBuilder is not synchronized (faster, single-threaded use).
- What happens on integer overflow? It silently wraps around (no exception) using two's complement arithmetic.